COMUNICA SERTÃO — GUIA RÁPIDO DE APRESENTAÇÃO

Roteiro de Fala Slide por Slide & Respostas Técnicas para a Banca

IFRS - Campus Sertão
Tecnologia em ADS
(2026)

Autor: Fabricio Cardoso de Abreu **Orientadora:** Caroline Guterres Silva **Foco:** Arquitetura Back-End, APIs & Dados

1. ROTEIRO DE FALA SLIDE A SLIDE (25 SLIDES)

SLIDE 01 Capa Oficial (Comunica Sertão - Arquitetura Back-End)

> "Olá a todos. Meu nome é Fabricio Cardoso de Abreu, sob orientação da professora Caroline Guterres Silva, e hoje apresento o desenvolvimento da infraestrutura de Back-End do projeto Comunica Sertão."

SLIDE 02 Contexto e Desafios de Engenharia (Gargalos Operacionais)

> "O nosso ponto de partida foi identificar que a prefeitura recebia demandas de forma desarticulada. Sem uma API central e sem um banco estruturado, havia perda de chamados, falta de protocolos oficiais e impossibilidade de auditoria dos atendimentos."

SLIDE 03 Proposta de Arquitetura de Retaguarda (Decisões de Projeto)

> "Para solucionar isso, projetamos um back-end leve e moderno com Python e FastAPI, utilizando SQLite gerenciado via SQLAlchemy. Isso garante processamento assíncrono rápido, custo zero de licenças para o município e soberania total sobre os dados públicos."

SLIDE 04 Objetivos de Desenvolvimento da API (Escopo e Metas)

> "Dividimos os objetivos do back-end em 5 pilares: criar as rotas RESTful, processar os webhooks do WhatsApp, persistir dados com segurança, implementar controle rígido de acesso com aprovação de servidores e gerar protocolos automáticos para os cidadãos."

SLIDE 05 Análise Comparativa de Arquiteturas (Soluções de Mercado)

> "Ao analisar soluções existentes, vimos que ferramentas como Fala.BR são monolíticas e complexas; plataformas como Colab.re e Octadesk cobram mensalidades por operador e guardam dados em nuvens privadas. O Comunica Sertão foi desenhado sob medida: código aberto, dados locais e custo zero."

SLIDE 06 Requisitos Funcionais da API (RFs - Mapeamento e Prioridades)

> "Aqui priorizamos o que o sistema executa: no topo (Prioridade Alta) estão a recepção de webhooks, o fluxo do bot, a emissão de protocolos e o login com aprovação. Em prioridade média e baixa, colocamos os uploads de fotos, filtros avançados e relatórios."

SLIDE 07 **Critérios de Qualidade e Restrições (RNFs - Desempenho e Segurança)**

> "Os requisitos não funcionais definem a qualidade técnica: exigimos tempo de resposta inferior a 200 milissegundos (RNF01), banco de dados relacional com integridade ACID (RNF02) e sanitização estrita de dados com schemas Pydantic (RNF03)."

SLIDE 08 **Mapeamento de Casos de Uso do Back-End (Atores e Serviços)**

> "Mapeamos dois fluxos centrais: do lado do cidadão, que interage via webhook no WhatsApp registrando chamados e consultando protocolos; e do lado do servidor, que acessa via rotas REST autenticadas para triar demandas e homologar novos acessos."

SLIDE 09 **Casos de Uso: Integração com o Cidadão (Webhooks e Bot)**

> "Este diagrama detalha a esteira do cidadão: o bot recebe a mensagem, conduz a máquina de estados, permite anexar evidências fotográficas e devolve imediatamente o protocolo oficial gerado."

SLIDE 10 **Casos de Uso: Gestão Administrativa (Serviços RESTful)**

> "Aqui vemos as operações restritas da prefeitura: autenticação institucional, triagem e alteração de status dos chamados (Kanban), e a função crítica de aprovação de novos cadastros pelos administradores."

SLIDE 11 **Modelo Conceitual de Dados (Entidades do SQLite)**

> "O nosso banco relacional foi estruturado em entidades bem definidas: Cidadão, Secretaria, Categoria, Funcionário, Anexo e o Histórico de alterações de status, garantindo rastreabilidade completa."

SLIDE 12 **Dicionário de Dados: Entidades Principais (Atributos e Regras)**

> "Destaco aqui duas tabelas centrais: a tabela funcionarios, com campo de status (Pendente, Aprovado, Bloqueado) e separação de cargos; e a tabela ocorrencias, com protocolo único (CMS-2026-XXXX) e campos de geolocalização e prioridade."

SLIDE 13 **Esquema Relacional e Cardinalidades (Mapeamento SQLAlchemy)**

> "Este é o esquema relacional gerado pelo SQLAlchemy no SQLite. Vemos relacionamentos 1 para N entre Cidadão e Ocorrências, e entre Ocorrência e o Histórico de Status, garantindo consistência por chaves estrangeiras."

SLIDE 14 **Pilha Tecnológica: FastAPI (Framework Web Assíncrono)**

> "Escolhemos o FastAPI por ser um dos frameworks mais rápidos do ecossistema Python. Ele opera sobre o padrão ASGI, permitindo alto volume de requisições concorrentes com baixíssima latência."

SLIDE 15 **Pilha Tecnológica: Python 3 & Uvicorn (Ambiente e Servidor ASGI)**

> "Utilizamos Python 3 com tipagem estática e o servidor Uvicorn. Essa combinação entrega uma execução assíncrona robusta, consumindo pouquíssima memória do servidor municipal."

SLIDE 16 **Pilha Tecnológica: SQLAlchemy & SQLite (ORM e Persistência)**

> "A camada de dados utiliza SQLAlchemy ORM e SQLite embutido (comunicasertao.db). Essa escolha garante transações atômicas (ACID) e simplifica drasticamente a manutenção e as rotinas de backup da prefeitura."

SLIDE 17 **Pilha Tecnológica: Pydantic Schemas (Validação de Contratos)**

> "O Pydantic é o responsável pela sanitização dos dados. Qualquer requisição com tipos corrompidos ou campos inválidos é barrada antes mesmo de tocar o banco de dados, prevenindo injeções maliciosas."

SLIDE 18 **Pilha Tecnológica: OpenAPI & Webhooks (Padronização e Integração)**

> "O back-end gera automaticamente a documentação interativa OpenAPI no /docs, facilitando os testes da API. Além disso, criamos o endpoint receptor para os webhooks da Meta Cloud API do WhatsApp."

SLIDE 19 **Implementação: Documentação Interativa (Swagger UI e Login)**

> "Na prática, a rota /docs permite testar todas as operações em tempo real. Implementamos a autenticação corporativa restrita ao domínio oficial @sertao.rs.gov.br, bloqueando acessos não autorizados."

SLIDE 20 **Implementação: Serviços de Ocorrências (Endpoints REST e JSON)**

> "Aqui vemos a estrutura de resposta JSON das ocorrências. Cada chamado trafega com seu protocolo, dados do munícipe, categoria, fotos e prioridade calculada."

SLIDE 21 **Implementação: Motor Conversacional do Bot (Webhooks e Protocolos)**

> "Implementamos a máquina de estados que processa as mensagens do WhatsApp. O cidadão é guiado por opções numéricas simples e, ao concluir, o sistema gera o protocolo anual sequencial."

SLIDE 22 **Implementação: Tratamento de Mídias e Uploads (Arquivos Locais)**

> "Criamos o endpoint /upload/foto que valida as extensões das imagens (JPG, PNG, WEBP), gera identificadores únicos via UUID e disponibiliza o acesso estático seguro para visualização no painel."

SLIDE 23 **Implementação: Controle de Acessos e Governança (Aprovação Admin)**

> "Esta é a camada de segurança administrativa: quando um novo servidor se cadastra, ele entra como 'Pendente'. Somente os perfis administradores cadastrados (Fabricio, Patrick e Prefeitura) têm autorização para aprovar o acesso."

SLIDE 24 **Matriz de Rastreabilidade da Arquitetura (RFs x RNFs)**

> "Essa matriz comprova a engenharia do projeto: cada Requisito Funcional desenvolvido atende rigorosamente a um critério de qualidade não funcional, seja em baixa latência, segurança ou integridade transacional."

SLIDE 25 **Perspectivas Futuras e Conclusão (Evolução da Infraestrutura)**

> "Como próximos passos, faremos a hospedagem definitiva do servidor em nuvem com SSL, a ativação do webhook oficial com o chip da prefeitura na Meta Cloud API e o disparo automático de notificações no WhatsApp do cidadão quando o chamado mudar de status. Concluímos com sucesso uma arquitetura sólida, segura e pronta para o município. Muito obrigado!"

2. PERGUNTAS CLÁSSICAS DA BANCA & RESPOSTAS TÉCNICAS PRONTAS

1. "Por que escolheram SQLite e não PostgreSQL ou MySQL?"

Resposta: Para a escala e realidade de Sertão (5.500 habitantes), o SQLite gerenciado via SQLAlchemy atende perfeitamente: possui suporte completo a transações ACID, não consome memória ociosa com processos de servidor em segundo plano e simplifica backups da prefeitura (basta copiar o arquivo .db). Além disso, como utilizamos o SQLAlchemy ORM, se no futuro o município precisar de PostgreSQL, basta alterar uma linha de configuração na string de conexão, sem reescrever o código da aplicação.

2. "Como a API garante que apenas funcionários autorizados acessem o sistema?"

Resposta: Através de uma dupla camada de segurança: primeiro, a rota de cadastro e login restringe obrigatoriamente e-mails institucionais oficiais (@sertao.rs.gov.br). Segundo, mesmo que qualquer pessoa tente se cadastrar, o status é gravado como "Pendente", e a API bloqueia a autenticação retornando código HTTP 403 (Proibido) até que um dos três Administradores homologados aprove manualmente o cadastro no painel.

3. "O que acontece se dezenas de cidadãos enviarem mensagens no WhatsApp ao mesmo tempo?"

Resposta: A arquitetura foi construída sobre FastAPI e o servidor ASGI Uvicorn, que opera com laço de eventos assíncrono (async/await). Quando múltiplos webhooks chegam simultaneamente, o servidor processa as requisições de forma não bloqueante, entregando tempos de resposta na faixa de dezenas de milissegundos sem sobrecarregar a CPU.

4. "Qual a diferença entre Requisitos Funcionais (RFs) e Não Funcionais (RNFs) no seu trabalho?"

Resposta: Os Requisitos Funcionais (RFs) representam o que a API realiza — receber mensagens, gravar ocorrências, emitir protocolos e autenticar servidores. Já os Requisitos Não Funcionais (RNFs) representam os critérios de qualidade e restrições técnicas — tempo de resposta inferior a 200ms, segurança de dados com Pydantic e integridade transacional com SQLite/SQLAlchemy.